

# Assignment 4

Aaron Bussche

David Magaram

Jason Kasdiran

Group 28

April 14, 2026

## Introduction

Evolutionary algorithms (EA) are a population-based optimization method. They are based on natural selection and work by exploring new solution areas and refining good already found solutions, where candidate solutions are improved by mutating and crossover.

By investigating the traveling salesman problem and string search, this project explores how different parameter configurations influence convergence speed, population diversity, and algorithm performance.

## Methods

This project will implement a genetic algorithm (GA) for string search and memetic algorithm (MA) for traveling salesman problem. This section describes the experimental setup and design choices.

### String search

A string search genetic algorithm was implemented and the used parameters are displayed in Table 1. "NaturalComputing" was used as the target string with a length  $L = 16$ , comprising of both lowercase and uppercase letters from an alphabet  $|\Sigma|$ . Since fitness is the number of characters that match the target, the maximum fitness is 16.

The algorithm aims to match the target string by evolving a population of  $N = 200$  individuals. Each generation follows a few simple steps. First, during the tournament selection phase, a random subset  $K$  is drawn from the population  $N$  and the fittest string is kept for reproduction.

Larger  $K$  values mean more potentially high-fitness strings, which can lead to the selection of only the best performers, working as a filter against weaker candidates. While it usually leads to faster convergence, it can also reduce diversity as the entire population will mimic the strongest strings.

Then, each pair undergoes single-point crossover with a probability  $p_c = 1$ , producing 2 new offsprings. Crossover selects a random index and swaps the parent's characters beyond that cutoff. Additionally, every character of the offspring has a small mutation rate  $\mu$  to randomly become one of the letters from the alphabet  $|\Sigma|$ . Finally, the entire parent generation is replaced with the new offspring generation.

| Parameter                   | Value                                                       |
|-----------------------------|-------------------------------------------------------------|
| String length $L$           | 16                                                          |
| Alphabet $ \Sigma $         | 52                                                          |
| Population size $N$         | 200                                                         |
| Tournament selection $K$    | $\{2, 5\}$                                                  |
| Crossover probability $p_c$ | 1                                                           |
| Mutation rate $\mu$         | $\{0, 1/L, 3/L\} + \text{sweep from } 0/L \text{ to } 10/L$ |
| Runs                        | 10                                                          |
| Max generations $G_{\max}$  | 100                                                         |

Table 1: Parameters for Exercise B1.

## Traveling Salesman Problem

For the TSP, both a simple evolutionary algorithm (EA) and a memetic algorithm (MA) were implemented. Each solution is encoded as a permutation of city indices representing a tour. The simple EA uses ordered crossover (OX): a random contiguous segment is copied from one parent and the remaining cities are filled in the order they appear in the second parent. Mutation is applied via swap mutation, where two randomly selected positions in the tour are exchanged. Tournament selection ( $K = 5$ ) is used to select parents, and generational replacement with no elitism is applied.

The memetic algorithm extends the EA by applying 2-opt local search to each individual after mutation. The 2-opt procedure iteratively reverses sub-sequences of the tour to eliminate crossing edges, repeating until no improving swap can be found. This Lamarckian approach allows individuals to locally optimise their fitness before evaluation, which typically accelerates convergence. Both algorithms were evaluated on the custom 50-city dataset and on the TSPLIB benchmark instance *eli51*.

| Parameter                   | Value                                  |
|-----------------------------|----------------------------------------|
| Population size $N$         | 100                                    |
| Tournament selection $K$    | 5                                      |
| Crossover probability $p_c$ | 0.8                                    |
| Mutation rate $\mu$         | 0.3                                    |
| Local search                | 2-opt                                  |
| Runs                        | 10                                     |
| Datasets                    | eli51, TSPLIB, Custom 50-city instance |

Table 2: Parameters for Exercise B2.

## Results

### B1

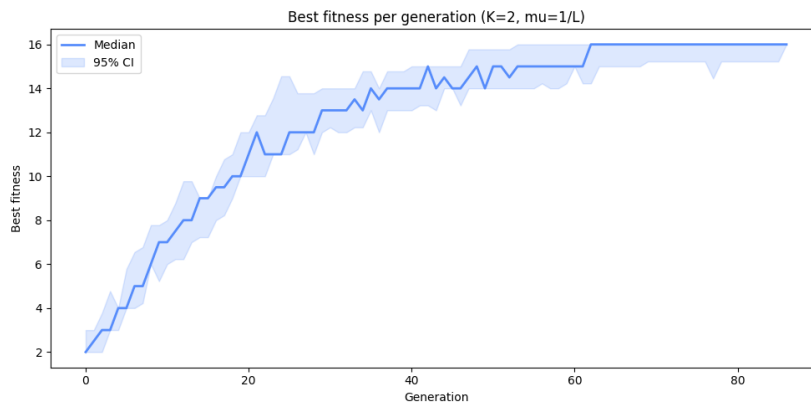


Figure 1: Results of running the GA 10 times with baseline values

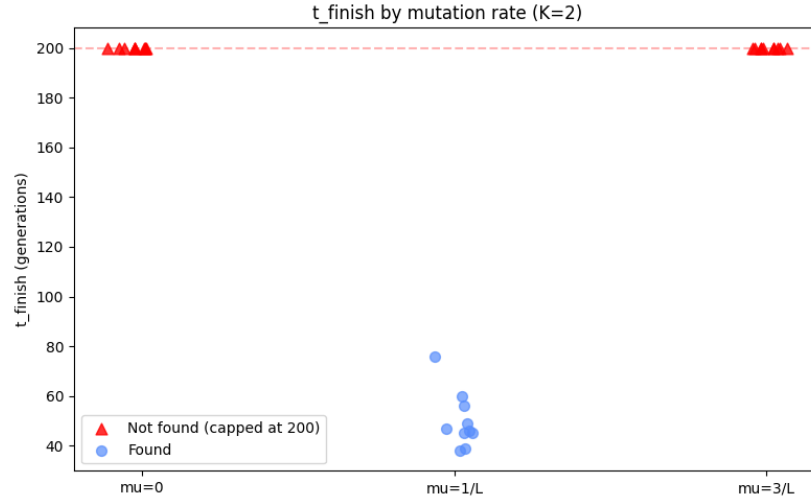


Figure 2: Three different mutation rates with  $K=2$ . Of these mutation rates, the GA manages to find the target only with  $\mu=1/L$  before getting cut off at 200 Generations

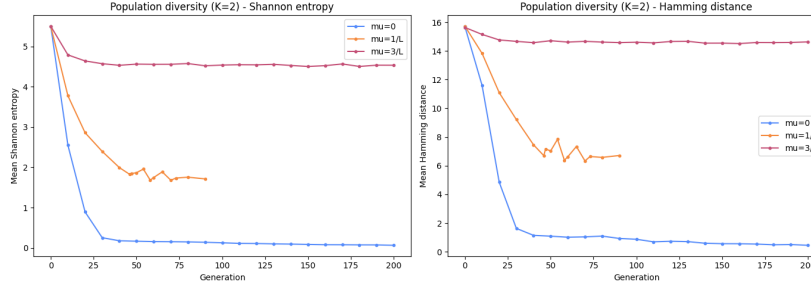


Figure 3: Measuring the diversity of the runs in two different ways, with Shannon entropy and Hamming distance. Both show that  $\mu=0$  leads to no diversity,  $\mu=3/L$  leads to high diversity through randomness while  $\mu=1/L$  manages to strike a middle ground (and reach the target).

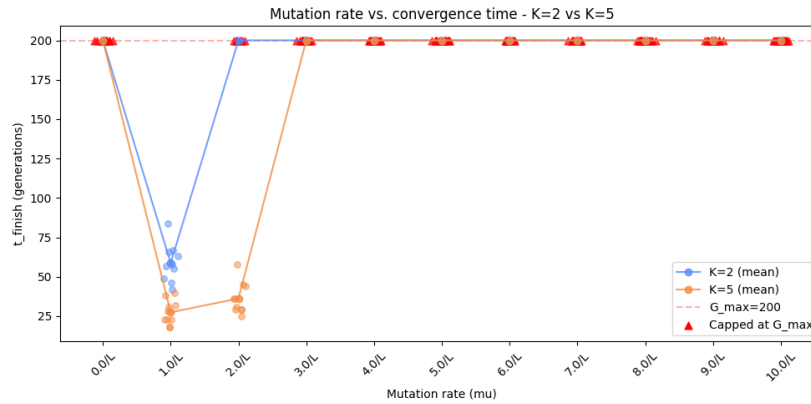


Figure 4: Higher tournament size increases selection pressure. We repeat the  $\mu$  sweep to see how this interacts with mutation rate.  $K=5$  leads to results faster & can also converge with  $\mu=2/L$ , though  $\mu=1/L$  remains optimal.

B2

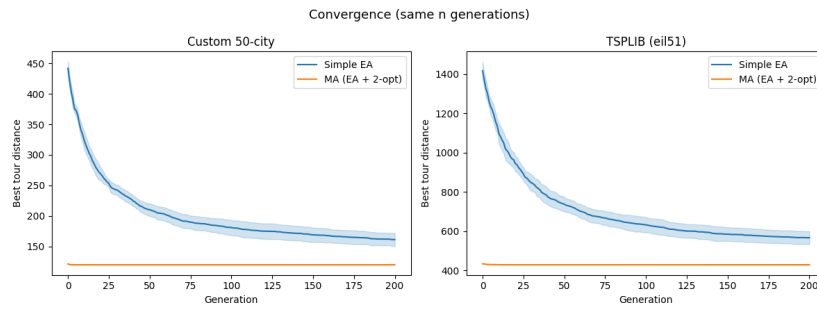


Figure 5: Both of the datasets show MA reaching better results in fewer generations, and in total.

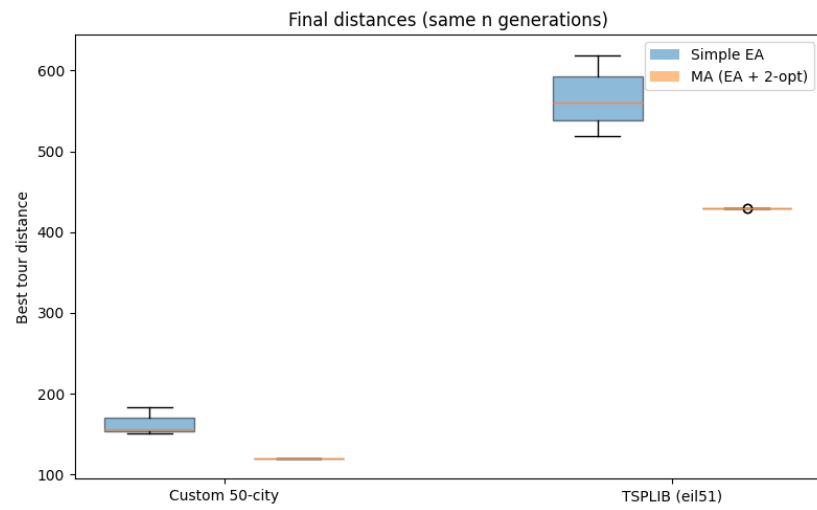


Figure 6: In this comparison, MA clearly beats EA in distance and consistency.

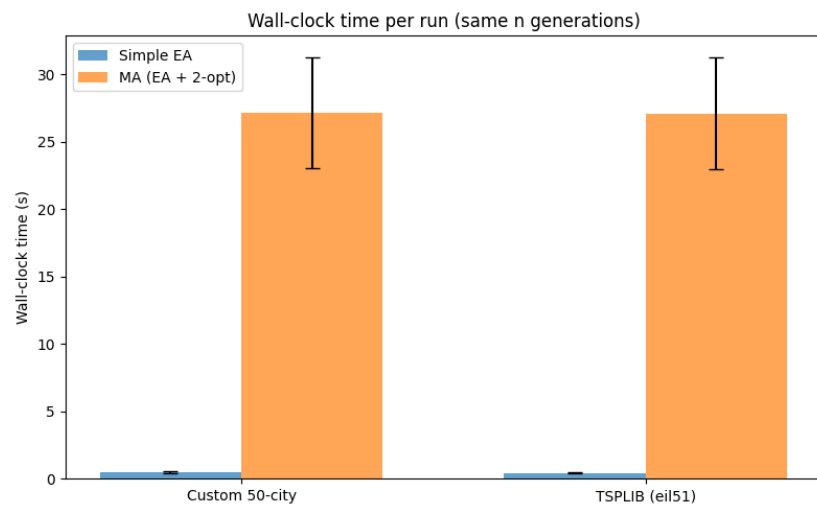


Figure 7: Per generation, MA takes much more compute & longer than EA. It is therefore not fair to compare them in this way.

We considered comparing them on number of operations, but the operations of MA are only in its neighborhood, so comparing them directly with whole generations of EA would again be unfair. We therefore decided to compare wall-clock time that, while influenceable by all kinds of factors and therefore not overly scientific, provides a good baseline for real-use performance, especially if averaged over 10 runs each.

Comparing the MA and simple EA on the same number of generations is not a fair baseline. Each MA generation includes a full 2-opt local search over every individual in the population, making it far more computationally expensive than a single EA generation. Any advantage observed per generation could therefore reflect additional computation rather than algorithmic superiority. We considered normalizing by number of fitness evaluations, but since 2-opt operates within a solution’s neighborhood its evaluation count is not directly comparable to a full population generation in the EA. We therefore compare on wall-clock time, which — while sensitive to system-level factors — provides a practical performance baseline when averaged over 10 runs.

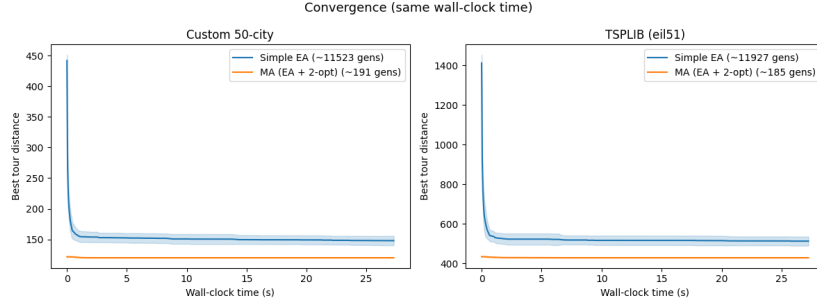


Figure 8: If compared instead on wall-clock time, MA still converges on better values, but EA’s curve also flattens to near its best result in a comparable time as MA. While EA goes through more than 11-thousand generations in this time, MA only reaches around 190.

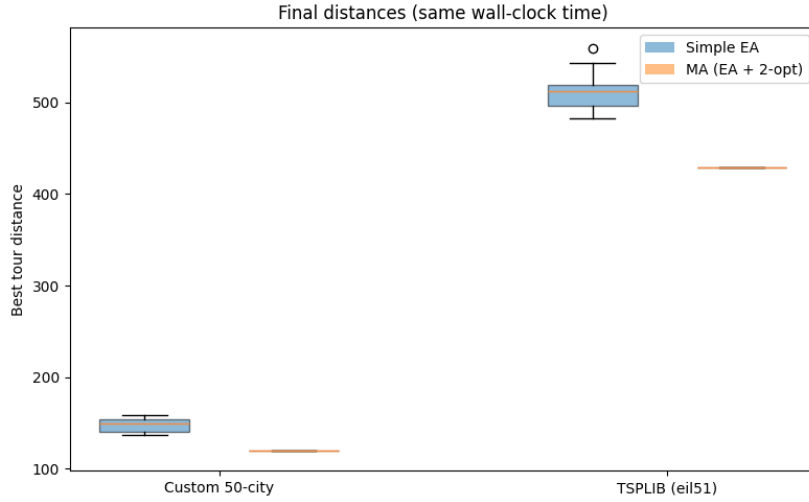


Figure 9: While the gap has shrunk considerably from comparing by number of generations, MA still provides better and more consistent results than EA.

Even under a fair wall-clock comparison, the MA consistently reaches shorter and more consistent tour distances than the simple EA on both datasets. The addition of local search therefore genuinely improves performance beyond what the extra computation alone can explain: 2-opt allows individuals to escape local optima that trap the plain EA, reducing both mean tour length and variance across runs.

**Conclusion**

**Discussion**

# Appendix

## Part A

### A1

What can you conclude about the effects of fitness scaling on selection pressure?

Adding constants raises all fitness all values by that amount. So it smooths out all the values. For example, when we compare f2 (which only squares X) and f4 (which squares X and adds 20), we can see that the probabilities of f4 are closer together. Probabilities are increased and reduced, so there is a much smaller gap between them, which also reduces selection pressure. Multiplying by a constant does not change the selection probabilities. If we compare f2 and f3, we can see that the probabilities stay the same even if we scale it. So scaling fitness does not have an effect on the selection pressure.

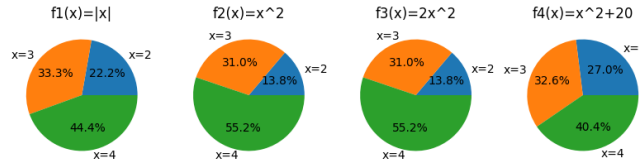


Figure 10: Selection probability of the three individuals for each function.

| x | f1 | p1     | f2 | p2     | f3 | p3     | f4 | p4     |
|---|----|--------|----|--------|----|--------|----|--------|
| 2 | 2  | 0.2222 | 4  | 0.1379 | 8  | 0.1379 | 24 | 0.2697 |
| 3 | 3  | 0.3333 | 9  | 0.3103 | 18 | 0.3103 | 29 | 0.3258 |
| 4 | 4  | 0.4444 | 16 | 0.5517 | 32 | 0.5517 | 36 | 0.4045 |

Table 3: Fitness function value and selection probability for each individual and each function.

### A2

Does the algorithm find the optimum? Yes. It is able to reach fitness 100, meaning all 100 bits were successfully set to 1.

What do you see? The version without selection is stuck at around 50 and never converges. The version with selection keeps steadily climbing, ultimately reaching 100.

Can you conclude anything on the difference between these two versions based on the above results? If yes, why? If not, what should you do to make a fair comparison? No. We would need multiple runs as EAs are stochastic and each run produces different results because of initialization and random mutation. To make it fair, we should run each version multiple times and use averages.

Is there a difference in performance? Yes. It is apparent that there is a difference in performance between the versions. Across all the runs, the version with selection always converges to fitness 100 while the version without selection is always near 50 through all the 1500 generations. The algorithm has no memory of good solutions and cannot improve.